START HERE

Your Verilog → a checked chip layout

Bring your working Verilog design. Leave with a tested Tiny Tapeout layout.

Your Verilog
existing design

→

Connect pins
Tiny Tapeout wrapper

→

Test
check the behaviour

→

Build layout
GDS + checks

→

Save evidence
ready for review

Recommended learning order • test, docs and gds workflows start independently.

01 Open the HTML guide and the starter ZIP beside GitHub.

02 Follow guide pages 2–14, including GLS. Tick each checkpoint.

CHECK I have opened the guide, signed in to GitHub and found the starter ZIP.

1. Start with the design you already have

Eight stored bits. One input bit. One clock edge at a time.

01 Extract the starter ZIP. Open src/shift_register.v.

02 Reset clears; enable shifts; otherwise hold.

03 Shift 1, 0, 1 in the HTML clock experiment.

src/shift_register.v

```
    always @(posedge clk) begin
        if (rst)
            q <= 8'b00000000;
        else if (enable)
            q <= {q[6:0], serial_in};
    end
endmodule
```

Excerpt • full file in the guide and example download

CHECK I know my circuit’s reset polarity, shift direction and enable behaviour.

1. Start with the design you already have — in practice

PAUSE & PREDICT

Starting at 00000000, what do inputs 1, 0, 1 produce after three enabled edges?

Answer: 00000001 → 00000010 → 00000101. A disabled edge keeps 00000101.

CHECK I know my circuit’s reset polarity, shift direction and enable behaviour.

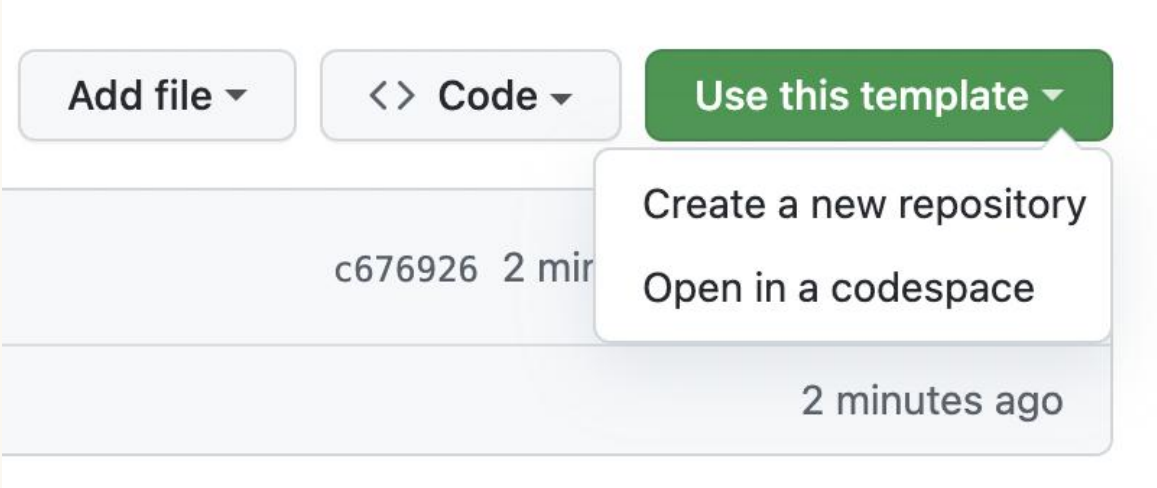
2. Make your own Tiny Tapeout project

Make your own copy of the official SKY Verilog template.

01 Open TinyTapeout/ttsky-verilog-template.

02 Use this template → Create a new repository.

03 Choose your account. Name: tt-shift-register. Visibility: Public.



GitHub's official screenshot shows the template menu. Use this control on the SKY template page.

CHECK My repository belongs to my account and contains src, test, docs and info.yaml.

Open reference ↗

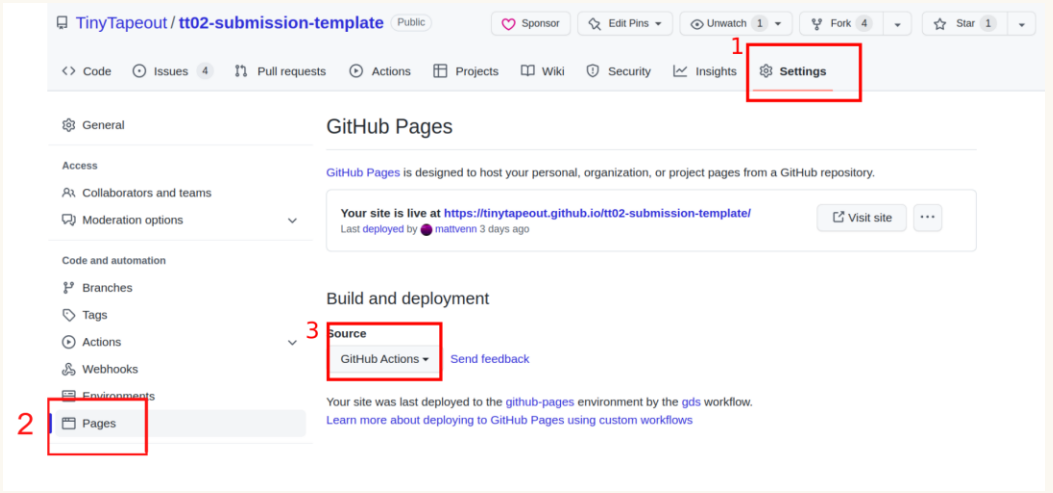
3. Turn on the build and results page

Actions runs your tools. Pages shows the layout.

01 Open Actions. Enable workflows if asked.

02 Settings → Pages → Source: GitHub Actions.

03 Keep .github, src/config.json and test/requirements.txt.



1 Settings → 2 Pages → 3 GitHub Actions. Official FAQ screenshot; step 2 annotation added.

CHECK Actions is available and Pages → Source says GitHub Actions.

Open reference ↗

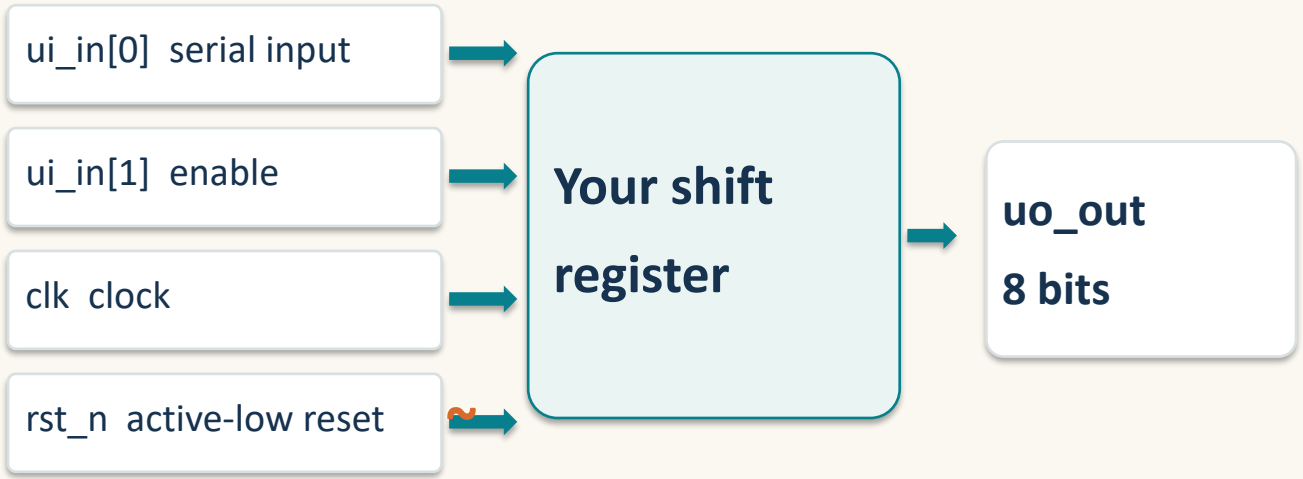
4. Plan how your signals reach the chip pins

A wrapper connects your signal names to Tiny Tapeout's fixed ports.

01 ui_in[0] → serial input. ui_in[1] → shift enable.

02 Clock connects directly. Invert rst_n to make the core reset.

03 q → uo_out. Set unused uio_out and uio_oe to zero.



CHECK I can trace every used chip pin to a signal in my existing design.

4. Plan how your signals reach the chip pins

PAUSE & PREDICT

Is ena the same signal as our shift enable?

Answer: No. ena selects this project on the shared chip. Our shift enable is ui_in[1]. This core uses only ui_in[1] to decide whether to shift.

CHECK I can trace every used chip pin to a signal in my existing design.

5. Add the wrapper around your design

Keep the existing circuit. Add one small module around it.

01 Replace src/project.v with the supplied wrapper.

02 Keep tt_um_student_shift_register for this exercise.

03 The instance below joins each circuit port to its chip signal.

src/project.v

```
    shift_register core (
        .clk       (clk),
        .rst       (~rst_n),
        .enable    (ui_in[1]),
        .serial_in (ui_in[0]),
        .q         (uo_out)
    );
```

Excerpt • full file in the guide and example download

CHECK My wrapper has the eight required ports and instantiates shift_register with the intended connections.

Open reference ↗

6. Tell both tools which files to use

Both tools need both files. Three top-module names must agree.

src/project.v
module name

→

info.yaml
top_module

→

test/tb.v
instance name

Both source lists: project.v + shift_register.v

01 One top name: tt_um_student_shift_register.

02 Two source lists: project.v and shift_register.v.

CHECK All three top-module names match; both source lists contain both Verilog files.

6. Tell both tools which files to use — in practice

Location	Must agree
src/project.v	module tt_um_student_shift_register
info.yaml	top_module: tt_um_student_shift_register
test/tb.v	tt_um_student_shift_register user_project
info.yaml source_files	project.v + shift_register.v
test/Makefile PROJECT_SOURCES	project.v + shift_register.v

CHECK All three top-module names match; both source lists contain both Verilog files.

7. Describe the design and its clock

Describe how to use the circuit. Keep its clock settings consistent.

01 info.yaml: your author name, title and correct pin labels.

02 docs/info.md: behaviour, test procedure and hardware needs.

03 50 MHz target = 20 ns per clock cycle.

Place	Clock setting
info.yaml	clock_hz: 50000000
src/config.json (template)	CLOCK_PERIOD: 20 (nanoseconds)
test/test.py cycle()	10 ns low + 5 ns to sample + 5 ns = 20 ns

CHECK My descriptions, pin map and clock settings agree with the actual design.

8. Replace the example test with your test

Test your shift register through its external pins.

01 Replace test/test.py with the supplied test.

02 Check reset, shift, hold, old data leaving and reset priority.

03 Compare each observed output with the expected value.

test/test.py — expected-value rule

```
# Shift the previous value and add the new bit.
expected = ((expected << 1) | bit) & 0xFF
assert await cycle(bit) == expected
```

CHECK My test checks my circuit’s behaviour rather than the template’s example.

8. Replace the example test with your test

PAUSE & PREDICT

After shifting 1 eight times from reset, what should the output be? What if enable then becomes 0?

Answer: 11111111. With enable = 0 it stays 11111111, even when serial_in changes.

CHECK My test checks my circuit’s behaviour rather than the template’s example.

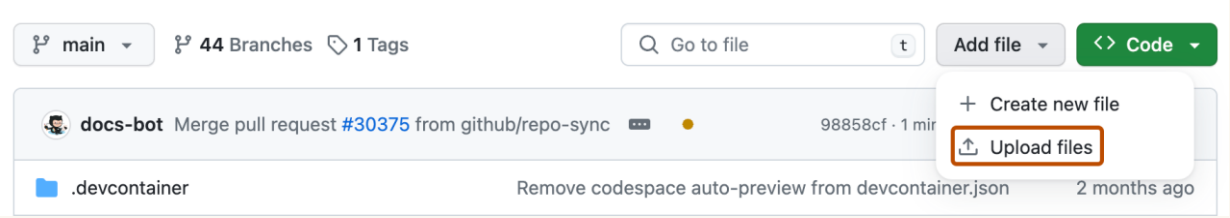
9. Upload the complete seven-file adaptation

Upload the contents of the extracted starter into your repository root.

01 Code → Add file → Upload files.

02 Drag info.yaml plus src, test and docs. Check all seven paths.

03 Commit to main: Add shift register. If needed, create and merge a pull request.



GitHub's official Add file screenshot locates the upload control. Upload into your repository root.

CHECK All seven files are saved at the intended paths, and the template infrastructure is still present.

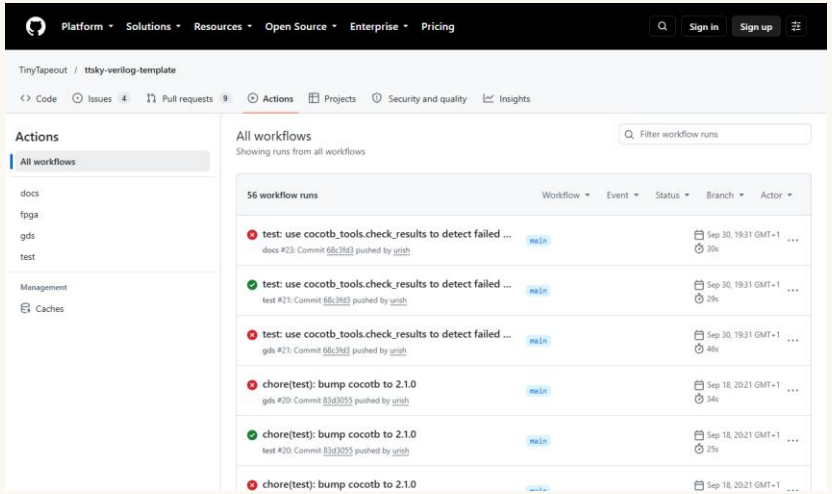
10. Check that your shift register passes

Read the test result for the commit containing all seven files.

01 Actions → test → your final commit's run.

02 Open the test job → Run tests. Zero failures or errors.

03 If it fails, fix the first error and save a new commit.



Official TEMPLATE Actions screen, for navigation only. Its red crosses are failed template runs, not your result. Your shift-register run has its own commit and result.

CHECK The RTL test passed for my exact final commit, with zero failures and errors.

10. Check that your shift register passes — in practice

PAUSE & PREDICT

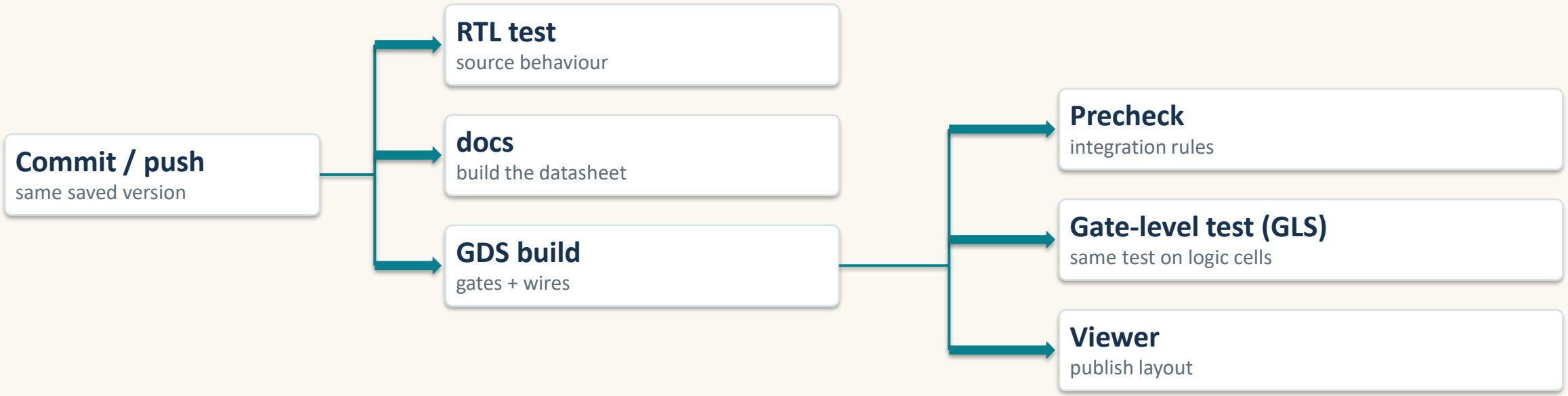
What does a passing test establish?

Answer: The adapted circuit passed the cases and assertions you wrote. It does not prove every possible behaviour, timing corner or manufactured chip.

CHECK The RTL test passed for my exact final commit, with zero failures and errors.

11. Build, check and look at the layout

Verilog becomes physical gates and wires. Check every required job.



Check every required job for one commit. The GLS walkthrough follows.

CHECK RTL, docs and every required gds job passed for one commit; I can open its layout and artifacts.

11. Build, check and look at the layout — in practice

Check	Plain meaning
Layout build	Place gates and connect them; inspect timing and geometry reports
Precheck	Check the design fits Tiny Tapeout’s integration rules
Gate-level test	Run your test against the generated gate connections
Viewer	Publish an image/browser view of the layout

Open the layout. Download tt_submission and GDS_logs.

CHECK RTL, docs and every required gds job passed for one commit; I can open its layout and artifacts.

12. See what changes for gate-level simulation

GLS runs your test against generated logic cells and their connections.

01 Keep the same test/test.py and external port checks.

02 Keep the GL_TEST fixture's VPWR and VGND connections.

03 The cloud job supplies the netlist and cell models; GATES=yes selects them.

test/tb.v — preserve this conditional connection

```
`ifdef GL_TEST
    .VPWR(VPWR),
    .VGND(VGND),
`endif
```

CHECK My test uses the wrapper ports, and the supplied fixture and gate-level Makefile branch are intact.

Open reference ↗

12. See what changes for gate-level simulation

RTL simulation	Gate-level simulation
Your source Verilog	Generated gate_level_netlist.v
Behaviour described in your code	Logic cells connected by the build
Same test/test.py	Same test/test.py
No extra cell power pins	GL_TEST fixture connects VPWR and VGND

01 Netlist = A generated description of logic cells and the wires joining them.

CHECK My test uses the wrapper ports, and the supplied fixture and gate-level Makefile branch are intact.

Open reference ↗

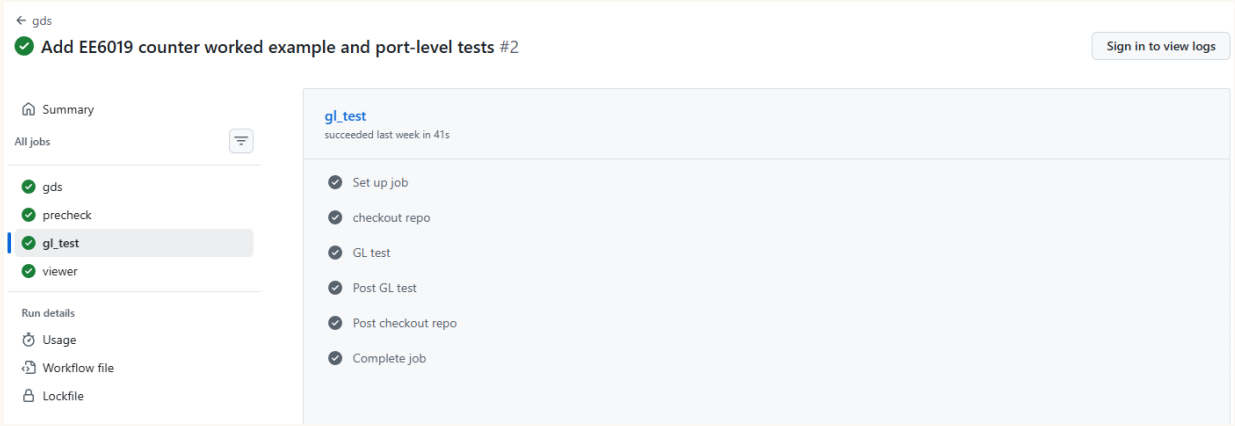
13. Open the GLS job and check its evidence

Open the gate-level job for the same commit as your RTL test.

01 Actions → gds → your run → gl_test → GL test → Run tests.

02 Read TESTS=1 PASS=1 FAIL=0 and confirm the whole job succeeds.

03 Summary → Artifacts → gatelevel_test_results: save results.xml and waveform.



Genuine successful gl_test job from the earlier COUNTER example at commit 61ef06b. It shows where to open GL test; it is not a shift-register result.

CHECK The gl_test job passed for the same commit; I saved its results XML and waveform.

Open reference ↗

13. Open the GLS job and check its evidence

PAUSE & PREDICT

Does this GLS result prove that every timing requirement is met?

Answer: No. This is a functional gate-level test. Inspect the build's separate timing reports as well; this walkthrough does not run an SDF timing-annotated simulation.

CHECK The gl_test job passed for the same commit; I saved its results XML and waveform.

Open reference ↗

14. Keep your result and use the guide again

Save one identifiable result: circuit, checks, layout and evidence.

01 Keep repository, commit and workflow links together.

02 Save the GDS, reports and datasheet for that same commit.

03 For your next design: map → wrap → list sources → test → build.

CHECK I have saved a checked layout package for one identifiable commit.

Open reference ↗

14. Keep your result and use the guide again

Unknown module / missing file

Check the three names and both source lists. Guide page 7.

Reset / shift assertion fails

Check pin bits, reset polarity and expected values. Pages 5 and 9.

Documentation fails

Complete metadata, sections and pin keys. Keep YAML indentation. Page 8.

Layout / viewer / gate test fails

Open the first failing job. See GLS on pages 13–14 and help on page 15.

CHECK I have saved a checked layout package for one identifiable commit.